Module 2 — Complete Explainer & Evaluation Reference

A full, plain-English walkthrough of Module 2 (NPC Behaviour and Coordination) — what it is, how every piece of it works, the exact code behind each claim, and the research grounding behind every design decision. Written for evaluation prep. Last updated 2026-08-09.

---

1. The Big Picture — What Module 2 Actually Is

Module 1 builds the building and decides where every character starts. Module 2 is responsible for what those characters — the terrorists holding the building, and the hostage — actually DO once the mission goes live: how they perceive the trainee, how they individually react, and how they coordinate as a group. Module 4 reads Module 2's telemetry to build the after-action report.

Think of Module 2 as three layers stacked on top of each other:

1. Individual brain — every terrorist and the hostage is its own finite state machine (FSM). A gunshot happens, they change state, they do something (take cover, run, shoot).
2. "Who reacts?" layer — when something happens (a gunshot, a door opening), the system has to pick WHICH NPC responds, not fire the event at everyone blindly. That is the NPCSelector scoring system.
3. Squad layer — terrorists in the same squad talk to each other: a Leader gives orders (converge, flank), and if one of them dies the alert spreads and the squad hunts as a group instead of everyone quitting on their own.

The original motivation for the whole squad-coordination layer (Section 7 below) was a specific, observed bug: a terrorist who saw the trainee and then lost sight of them would simply abandon the chase and walk back to its post — even if a squadmate was standing right there. Real react-to-contact doctrine says the opposite: a confirmed sighting should commit the WHOLE squad to a coordinated pursuit, not a private memory that quietly expires. That gap, and the U.S. Army doctrine that contradicts it, is the origin story documented in Assets/Docs/Literature_Review_Module2.docx and is why Section 7.4 exists.

2. Architecture Map — Files and Their Roles

File	Role
Assets/Scripts/Events/EventManager.cs	Central event bus — routes every ScenarioEvent
Assets/Scripts/Events/ScenarioEvent.cs	Immutable event data record
Assets/Scripts/Events/ScenarioEventType.cs	Enum of every event type the system knows
Assets/Scripts/NPC/TerroristController.cs	Terrorist FSM + combat + squad support brain (3,790 lines)
Assets/Scripts/NPC/HostageController.cs	Hostage FSM + follow/leverage logic
Assets/Scripts/NPC/PerceptionController.cs	Vision (FOV + raycast) and target-confirm timing
Assets/Scripts/NPC/HostageProfile.cs	Weak/Normal/Brave personality trait system
Assets/Scripts/NPC/AlertPropagator.cs	3-ring alert spreading (squad / proximity / none)
Assets/Scripts/NPC/Squad.cs	Squad membership, Leader directives, shared hunt/search logic

File	Role
Assets/Scripts/NPC/LeaderDirective.cs	Converge / Hold / Flank command data
Assets/Scripts/NPC/INPCResponder.cs	Interface every NPC implements to plug into the event system
Assets/Scripts/NPC/NPCRole.cs	Guard / Roamer / Leader / Hostage role enum
Assets/Scripts/NPC/TerroristState.cs, HostageState.cs	FSM state enums
Assets/Scripts/Selection/NPCSelector.cs	Multi-factor scoring — decides which NPC responds to a targeted event
Assets/Scripts/Telemetry/TelemetryLogger.cs	Central logging sink to JSON files for Module 4
Assets/XRI Starter Kit/.../PatrolLine.cs	Multi-waypoint patrol route follower
Assets/Scripts/Tests/Module2TestRunner.cs	13 automated integration tests (in-editor, Play mode)
Assets/Scripts/Tests/AblationExperimentRunner.cs	Generates CSV evidence the scoring formula's factors matter
Assets/Docs/Literature_Review_Module2.docx	The research grounding for every design decision below

3. The Event System — the Nervous System

`EventManager` is a singleton. Anything that happens becomes a `ScenarioEvent` (type, world position, instigator, timestamp, optional room/target IDs) and is passed to `EventManager.Instance.Raise(e)`.

3.1 Two routing modes, decided by one hardcoded list

Inside EventManager.cs there is a literal hash set that decides everything:

```
static readonly HashSet<ScenarioEventType> BroadcastTypes = new HashSet<ScenarioEventType>
{
ScenarioEventType.GunshotHeard,
ScenarioEventType.TerroristDown,
ScenarioEventType.RoomCleared,
ScenarioEventType.StressSpike,
};
```

Only these FOUR event types are "broadcast." Every other event type in the whole enum (RoomBreached, DoorOpened, AllyDownSeen, HostageContactStarted, TerroristHit, and so on) falls into the else branch — that is "targeted," and that is where NPCSelector scoring kicks in (Section 6).

- Broadcast events — delivered to EVERY eligible NPC. No competition, nobody picked, everyone who can respond does.
- Targeted events — delivered to the SINGLE best NPC, chosen by NPCSelector.SelectBest. Everyone else does nothing for that event.

3.2 Worked example — targeted event (RoomBreached)

Say the trainee walks into a room. RoomBreached fires. Three terrorists are eligible (CanRespond returns true for all three):

NPC	Role	Distance	State	LOS	Score
T1	Guard	4m	Idle	Clear	(1/4)x2 + 3.0x1 + 1.0x1 + 1.0x1 = 5.5
T2	Roamer	2m (closer!)	Idle	Clear	(1/2)x2 + 0x1 + 1.0x1 + 1.0x1 = 3.0
T3	Leader	3m	Suspicious	Blocked	(1/3)x2 + 0x1 + 0.8x1 + 0x1 = 1.47

T1 (the Guard) wins, even though T2 is physically closer — because "Guard" carries the role bonus specifically designed for RoomBreached (3.0). Only T1 gets RespondTo(e) called on it. T2 and T3 do nothing for this event. That is the whole point of scoring: not "nearest NPC wins," but "which NPC is the RIGHT one to send," using distance + role fit + how busy they already are + whether they can even see it.

3.3 Worked example — broadcast event (GunshotHeard)

No scoring at all here. EventManager loops the entire NPC registry and calls RespondTo(e) on EVERY NPC where CanRespond returns true:

```
foreach (var npc in registry)
if (npc.CanRespond(e)) { npc.RespondTo(e); responders.Add(npc); }
```

If 5 terrorists are within hearingRange (20m default), all 5 go Idle -> Suspicious simultaneously — nobody competes for "who gets to react."

3.4 The nuance: the SAME scoring formula gets reused after a broadcast

Right after a GunshotHeard broadcast, EventManager does a SECOND, separate step — it calls NPCSelector.SelectBest again, but only on the subset of terrorists that just reacted, to pick ONE of them to physically walk over and investigate the exact gunshot location:

```
var terrorists = responders.FindAll(r => r is TerroristController);
var investigator = NPCSelector.SelectBest(terrorists, e, false);
investigator?.InvestigatePosition(e.Origin, allowEscalation: true);
```

TerroristDown works the same way, narrowed even further to only Alert-state terrorists among the responders. It is not a second scoring SYSTEM — it is the same function, called twice, with two different-sized candidate lists: once implicitly for ordinary targeted events, once explicitly for "who gets sent to check it out" after a broadcast. RoomCleared and StressSpike have no such secondary step — pure broadcast, nothing scored, ever.

Worked example: 3 terrorists heard a shot and all went Suspicious (broadcast, no competition). Then the investigator pick scores just those 3:

NPC	Role	Distance	State	LOS	Score
T1	Roamer	8m	Suspicious	Blocked	(1/8)x2 + 2.0x1 + 0.8x1 + 0x1 = 3.05
T2	Guard	3m (closest)	Suspicious	Blocked	(1/3)x2 + 0x1 + 0.8x1 + 0x1 = 1.47
T3	Leader	5m	Suspicious	Clear	(1/5)x2 + 0x1 + 0.8x1 + 1.0x1 = 2.2

T1 wins — despite T2 being closest — because Roamer specifically carries the role bonus for GunshotHeard (2.0). T1 is dispatched to investigate; T2 and T3 stay Suspicious where they are.

3.5 Event derivation (automatic, inside Raise)

- ShotFired always also raises GunshotHeard.
- 3+ ShotFired events within 2 seconds automatically raises StressSpike (tracked with a rolling Queue<float> of recent shot timestamps in EventManager). This is what escalates hostages to Panic during sustained gunfire with no scripted trigger.

4. Terrorist FSM (TerroristController.cs)

States: Idle -> Suspicious -> Alert -> Engage, plus TakeCover, Retreat, Down.

4.1 Transition table

From	To	Trigger
Idle	Suspicious	GunshotHeard within hearingRange (20m default)
Suspicious	Alert	PlayerSeen (own perception), RoomBreached, DoorOpened, another GunshotHeard, or AllyDownSeen/TerroristDown
Alert	Engage	TargetConfirmed — sustained LOS for targetConfirmTime (0.5s), from PerceptionController
Engage	Alert	PlayerLost — LOS broken for lostSightDelay (1.5s)
Engage	Retreat	TakeHit() drops health <= retreatHealthThreshold (40), once per life only, never for a hostage guardian
any active	Down	health <= 20 (checked inside TakeHit)
Alert	TakeCover	Manual cover-seeking logic (MoveTocover); falls back to Alert if no cover found
Retreat	Alert	Automatic, after holding a fallback point for retreatHoldTime (4s) — re-arms perception so re-engagement is possible
Suspicious	Idle	SuspiciousTimeout coroutine, 6s of nothing new

From	To	Trigger
Alert	Idle	AlertGiveUpTimeout coroutine, 12s with no re-acquisition — BUT gated by the squad hunt, see 4.3

4.2 Perception is personal, not broadcast

PlayerSeen, TargetConfirmed, and PlayerLost are NOT routed through the shared event bus's CanRespond/RespondTo path — that path explicitly rejects them. Instead, PerceptionController (a sibling component doing FOV cone + raycast checks every 0.2s) calls HandleDetection(e) directly on its own TerroristController. This matters: an NPC that never personally saw the player can never be handed a squadmate's sighting and start shooting blind — only genuinely "shared" information (Section 7.4) is allowed to spread.

4.3 The "no casual give-up" fix

This is the direct code fix for the bug that motivated the literature review. Entering Idle is intercepted: if this NPC personally confirmed the player, OR its squad is still actively hunting (Squad.HuntActive), it is bounced straight back to Alert instead of settling down. The only sanctioned way to stand down is Squad.EndHunt() — a collective, squad-wide decision (Section 7.4).

4.4 Combat: health, damage, morale, positioning

- maxHealth = 100. TakeHit(damage) reduces it; <=20 -> Down.
- Escalation/morale accumulator (_escalation): rises from ally-down sightings (+2), stress spikes (+1), being hit (+1), and a periodic tick every 8s of sustained combat. Once it crosses escalationThreshold (3), CombatPosture flips ONE-WAY from HoldAndShoot to CoverAndPeek.
- HoldAndShoot: holds a firing-ring standoff distance (steps back within 3m, advances past ~3.5m — it visibly presses the attack).
- CoverAndPeek: claims the best nearby CoverPoint, holds fire ~3.5s, then relocates — classic peek-and-bound suppression behaviour.
- Death (EnterDownState): stops shooting, drops any held hostage, cancels all coroutines, disables PatrolLine/NavMeshAgent, snaps the body to the floor, disables hitboxes (and optionally non-trigger colliders so live NPCs can walk over corpses), plays the death animation, raises TerroristDown.

4.5 Key tunables (defaults)

hearingRange=20m, wanderRadius=12m, retreatHealthThreshold=40, retreatHoldTime=4s, flankOffset=5m, responseCooldown=4s, minStandoffDistance=3m, preferredStandoffDistance=3.5m, escalationThreshold=3, coverHoldTime=3.5s, suspiciousTimeout=6s, alertGiveUpTime=12s.

5. Hostage FSM (HostageController.cs)

States: Calm, Fearful, Freeze, Panic, Follow, Held, Freed, Down.

5.1 Transition table

From	To	Trigger
Calm	Fearful	GunshotHeard beyond panicDistance (6m)
Calm	Panic	GunshotHeard within panicDistance (6m), or a misread of a distant shot, or StressSpike
Fearful	Panic	TerroristEnteredRoom, or StressSpike
Fearful	Freeze	GunshotHeard within freezeRange (5m)
Panic	Freeze	Panic sustained >= freezeThreshold (3s), via SustainedThreatCheck
Panic	Fearful	TerroristDown (threat eliminated) — unless misread
Follow	Fearful ("regression")	Close GunshotHeard during escort — flagged [REGRESSION] in telemetry
any (except Freed/Follow)	Calm	RoomCleared — unless misread
any non-terminal	Follow	HostageContactStarted — unless misread as a threat (-> Freeze instead)
-	Freed	HostageFreed event, or auto-detected when runawayController reports IsHiding
-	Held	SeizeAsLeverage(captor) by a guardian terrorist
Held	Fearful	ReleaseFromHold()
-	Down	TakeHit to 0 health, or guardian Execute() — terminal, raises OnKilled (mission FAIL)

5.2 Personality profiles (HostageProfile.cs)

Three profiles: Weak, Normal, Brave. ALL THREE use identical distance thresholds — the code comments explain why: published dB-to-distance research shows gunfire exceeds the fear/panic loudness threshold at every realistic in-building range, so distance literally cannot discriminate between personalities at close quarters. The only differentiator is ThreatDiscrimination (0..1):

Profile	Discrimination	Spawn weight
Weak	0.35	10.9%
Normal	0.85	21.4%
Brave	1.00	67.7%

MisreadsNonThreat() rolls Random.value > ThreatDiscrimination at four decision points (grading a distant shot, calming down after the threat is gone, calming down on RoomCleared, recognising an approaching rescuer). Lower discrimination means more overreaction/underreaction, which naturally makes a Weak hostage slower to extract WITHOUT any hardcoded delay. Spawn weighting matches published trauma-trajectory prevalence (Galatzer-Levy, Huang & Bonanno 2018), renormalised over 3 profiles.

The HONEST status, straight from the code comments in HostageProfile.cs: "The MECHANISM and its DIRECTION are research-grounded. The three discrimination values themselves are design calibrations, not published values — the literature supplies no numeric probability. Treat them as tunable, and prefer a sensitivity analysis (showing the Weak > Normal > Brave ordering is stable across a range) over defending any exact value." This exact honesty pattern is the template used later for defending the NPCSelector role bonuses (Section 6.7).

5.3 Key tunables (defaults)

panicDistance=6m, freezeRange=5m, freezeThreshold=3s, followSpeed=2.2, maxHealth=90 (three 30-damage rifle rounds = death, matching enemy fragility), injuredThreshold=45.

6. NPCSelector — "Who Responds?" (Selection/NPCSelector.cs)

For every targeted event, all eligible candidates (already filtered by CanRespond — not Down, not committed to another target, not on cooldown) are scored with this formula:

score = (1 / distance) x DistanceWeight (2.0)

+ roleBonus[role][evtType] x RoleWeight (1.0)

+ stateReadiness x StateWeight (1.0)

+ losBonus x LosWeight (1.0)

- Distance term: closer = higher score (inverse distance). Unbounded as distance -> 0.
- Role bonus table: Guard+RoomBreached=3.0, Roamer+GunshotHeard=2.0, Leader+AllyDownSeen=1.5, else 0.
- State readiness (0-1): Idle=1.0, Suspicious=0.8, Alert=0.5, TakeCover=0.2, Engage=0.0 — idle NPCs are preferred responders.
- LOS bonus: a raycast from the candidate's eye height to the event origin — 1.0 if clear, 0.0 if blocked.
- Candidates beyond MaxRange (30m) are excluded outright.

SetAblation(distance, role, state, los) zeroes out any subset of weights — the mechanism the project's ablation experiments drive to produce evidence for each factor mattering. Every decision is logged via TelemetryLogger.LogDecision.

6.1 Verifying the technique against real literature

Rather than trust the claim "this is standard utility-based AI" at face value, each source cited for it was checked directly. Here is the honest strength rating for each one.

6.1.1 Weighted Sum Model / Simple Additive Weighting (Triantaphyllou, 2000) — Strong

This is the real, formal name for "weight each factor and sum" — a canonical multi-criteria decision-making (MCDM) method, also called the weighted sum model (WSM) or simple additive weighting (SAW). Its textbook definition is explicitly TWO steps, not one: NORMALISE every criterion onto a common scale (typically via max-normalisation), THEN multiply by weight and sum. Multiple independent sources confirm normalisation is treated as mandatory, not optional polish, precisely "to prevent objectives with relatively large values from dominating the calculation."

6.1.2 Dill & Mark's Utility AI / IAUS (GDC 2010, Game AI Pro 3 ch.13 — Lewis) — Strong

The documented process is explicitly: (1) get raw input -> (2) normalise to 0-1 -> (3) apply a response curve -> (4) weight and accumulate. Their own worked example: a target at 25m out of 100m max range gives a raw ratio of 0.25, which then gets passed through a curve. Distance is NEVER used as a raw, unbounded 1/distance value in the reference implementation — it is normalised first, every time.

6.1.3 Multi-Robot Task Allocation survey (ACM Computing Surveys, 2024) — Adequate

Good support for the general factor set (distance + capability + availability), and explicitly confirms modern MRTA formulas normalise these metrics before combining. Supports "these are the right factors," does not rescue an unnormalised implementation.

6.1.4 CAD (emergency dispatch) systems — Weak / anecdotal

Real-world practice, not a citable formula. Useful as "here is a real-world analogy," not as mathematical grounding — say so if pressed.

6.1.5 Weapon-Target Assignment problem — Weaker than commonly implied

WTA is a nonlinear COMBINATORIAL optimisation over probability-of-kill x target-value across many simultaneous weapon/target pairs — a genuinely different problem structure from "score N candidates for one event, take the max." It shares the THEME of distance+capability, not the mechanism. Concede this is a loose analogy if an evaluator pushes on it specifically.

6.1.6 Dill & Martin (2011 I/ITSEC), "A Game AI Approach to Autonomous Control of Virtual Characters" — Weaker than commonly implied

Real, verified paper — but it is about "Angry Grandmother," a single scripted mixed-reality NPC combining Behaviour Trees + utility scoring for its OWN decisions, in a military training scenario. It is not a multi-agent best-responder selection algorithm. It supports "utility-based AI is used in real military training sims" as a general claim — it does not validate this specific 4-factor selection architecture.

6.2 The concrete flaw: the formula is not normalised

Plug real numbers into score = (1/dist)x2.0 + roleBonusx1.0 + stateReadinessx1.0 + losBonusx1.0:

Distance	Distance term alone	Max possible from role+state+LOS combined
0.2m	10.0	5.0
0.5m	4.0	5.0
1m	2.0	5.0
25m	0.08	5.0

At 0.2m, the distance term ALONE is double the maximum possible score from every other factor combined — role, state, and LOS become mathematically irrelevant. At 25m it is nearly zero — role/state/LOS become the entire decision. "DistanceWeight=2.0" does not mean "distance counts twice as much" the way it reads; it means distance silently swings between total dominance and total irrelevance depending on absolute proximity. That is the exact failure mode the normalisation step in WSM/SAW and utility-AI exists to prevent.

6.3 The corrected formula — properly grounded, same design intent

Normalise every term to [0,1] first (this is now literally the textbook Simple Additive Weighting procedure, citable directly to Triantaphyllou 2000, and it also correctly matches Dill/Mark/Lewis's 4-step process instead of just gesturing at it):

distanceScore = 1 - clamp(distance / MaxRange, 0, 1) // 1.0 close, 0.0 at MaxRange(30m)

roleScore = roleBonus / 3.0 // 3.0 was the max raw bonus -> now 0-1

stateScore = stateReadiness // already 0-1, unchanged

losScore = losBonus // already {0,1}, unchanged

score = 0.4 x distanceScore + 0.2 x roleScore + 0.2 x stateScore + 0.2 x losScore

Weights sum to 1 — the canonical WSM form — and 0.4/0.2/0.2/0.2 preserves the original intent that distance matters roughly twice as much as each of the other three, just correctly, on a comparable scale. This is not a cosmetic change — it fixes a real methodological gap, and makes the citation to Dill/Mark/Lewis STRONGER, not weaker, because the code now actually does what those sources describe rather than approximating it. As of this writing the fix is a documented recommendation; confirm with the project whether NPCSelector.cs has been patched to this corrected formula before citing it as implemented.

6.4 Extra nuance: the specific numbers, or comparing bonuses to each other?

The role-bonus values 3.0/2.0/1.5 never actually compete against each other in any single decision — Guard's bonus only applies when scoring RoomBreached, Roamer's only for GunshotHeard, Leader's only for AllyDownSeen. If an evaluator asks "why is Guard's bonus twice as big as Leader's?" the honest answer is: there is no principled reason they should even be comparable to each other. What DOES need defending is each number against the OTHER THREE terms it competes with inside its own decision — see 6.5.

6.5 Defending the role-bonus magnitudes — a real methodology, not a citation

Two separate questions, and they need two different defenses.

Question 1: why does a role bonus exist at all for Guard/RoomBreached, Roamer/GunshotHeard, Leader/AllyDownSeen?

This part IS documented — just not in academic literature, in the project's own code. NPCRole.cs states the design contract directly: "Guard covers doors and choke-points; prioritised for RoomBreached. Roamer patrols open areas; prioritised for GunshotHeard. Leader commands squad; prioritised for AllyDownSeen." That is a legitimate design rationale nobody can attack.

Question 2: why is it 3.0, and not 2.5 or 5.0?

This part has zero grounding, and faking a citation for it is the fastest way to lose credibility with an evaluator who checks. Defend it structurally instead, via a bounded-influence argument: with the current raw formula, the max possible score from role bonus is 3.0, out of a combined max of role+state+LOS = 5.0 (excluding distance). So role bonus can contribute up to 60% of the non-distance decision — meaningful, but it cannot single-handedly guarantee a win if the Guard is far away with no LOS while a Roamer is close with clear sight. That IS the correct design property: role-fit should be a strong preference, not a hard rule that overrides physical reality. This is precisely the argument Dill & Mark make for why utility AI uses multiple weighted considerations instead of hardcoded priority rules in the first place. With the corrected normalised formula

(6.3), this becomes even cleaner to state: role fit is capped at 20% of the total decision, so a Guard on the far side of the building with no line of sight cannot out-rank a Roamer standing right at the breached door.

6.6 A gap worth knowing before an evaluator finds it

The project's weight-sensitivity study (MODULE2_WEIGHT_SENSITIVITY_REPORT.md, referenced from the literature review postscript) sweeps each factor's WEIGHT (DistanceWeight/RoleWeight/StateWeight/LosWeight — the single outer multiplier) across a 0-4x range. It does NOT sweep the INNER table values (3.0/2.0/1.5) inside GetRoleBonus. So there is empirical evidence that "how much the role factor matters overall" is stable — but zero evidence, empirical or literature-based, for the specific 3.0/2.0/1.5 numbers themselves. If an evaluator asks "did you test the bonus VALUES themselves, not just the weight?" — the honest answer, at the time of this document, is no.

Two ways to close this gap: (Option A) simplify to a single flat "role match" bonus (everyone gets 1.0 if their role matches the event, 0.0 otherwise) — trivially defensible, removes the un-cited distinction entirely, loses a little nuance. (Option B) keep the differentiated bonuses but extend the ablation infrastructure to sweep the role-bonus TABLE VALUES themselves (not just the outer weight), and show the qualitative outcome (Guard preferred for room breaches, etc.) is stable across a reasonable range — the same honest framing HostageProfile.cs already uses for its discrimination values (Section 5.2).

7. Squad Coordination (Layer 3) — Full Detail With Worked Numbers

Setting the scene for every example below: a squad called "alpha" — one Leader, two Guards, patrolling a building. The trainee sneaks in and gets spotted by one Guard at time t=0s.

7.1 Alert propagation — three rings (AlertPropagator.cs)

Ring	Who	Delay	Needs line-of-sight?	What happens
Ring 1	Same squad (alpha)	0.3s	No	Idle/Suspicious members escalate to Alert
Ring 2	Anyone else within 15m (alertPropagation Radius)	1.0s	Yes	Escalates to Suspicious only
Ring 3	Everyone else	-	-	Nothing

Worked example: Guard_A spots the trainee at t=0s. At t=0.3s, the Leader and Guard_B (same squad) both flip to Alert — they do not need to see anything, they just got the "radio call." Meanwhile an unrelated squad's Roamer standing 10m away with a clear sightline flips to Suspicious at t=1.0s — a full second later, and only to "suspicious," not full alert, because it was not their own squadmate who called it in. A different Roamer 40m away (beyond the 15m Ring 2 radius) does nothing at all — that is Ring 3.

Special case: if a squad member DIES (not just spots someone), Ring 1 members who have direct line-of-sight to the body react with 0-second delay and jump straight to Engage — no 0.3s wait. Seeing a friend get shot is treated as more urgent than a routine sighting report.

7.2 Leader directives — Converge and Flank

- Leader -> Alert issues Converge. Every non-engaged squad member paths toward the threat, stopping at a standoff ring around 3.5m away (preferredStandoffDistance), so they arrive spread around the trainee rather than clumping on top of them.

- Leader -> Engage issues Flank. Each member computes its own side to approach from — offset 5m perpendicular (flankOffset) to the direction they are approaching from, picking whichever side they are naturally already closer to. No one is assigned "you go left, you go right" — it falls out of simple geometry.

Worked example: Leader spots the trainee at position (10, 0). Guard_B is approaching from the west, Guard_C from the east. When the Leader engages, Guard_B's flank point becomes roughly (10, -5) and Guard_C's becomes (10, +5) — they naturally end up on opposite sides of the trainee, 10m apart from each other, while the Leader holds the direct front. The "pincer" look is not scripted, it is offset-perpendicular geometry from each member's own position.

7.3 Leader succession

If the Leader dies mid-fight, Squad.PromoteNewLeaderIfNeeded runs automatically: it picks a Roamer if one is alive (preferred), otherwise any surviving non-guardian member, and calls AssumeLeadership() on them. Example: Leader dies at t=30s mid-engagement -> within the same frame, Guard_B (if it is the Roamer) becomes the new Leader and can issue directives from then on. Without this, killing the Leader first would permanently disable squad coordination for the rest of the mission — a cheap exploit the promotion logic closes.

7.4 The shared hunt — the fix for the literature-review bug, in full

This is the direct implementation of the literature review's central finding (Sections 2-4 of Literature_Review_Module2.docx: "a confirmed sighting is a trigger for coordinated pursuit, not withdrawal"). Real numbers, straight from Squad.cs:

```
const float HuntBudgetSeconds = 45f; // total time the squad presses a lost contact

const float SearchGrowthPerSec = 0.6f; // how fast the search circle grows

const float SearchRadiusMin = 5f; // starting radius

const float SearchRadiusMax = 18f; // cap


float radius = Mathf.Clamp(SearchRadiusMin + elapsed * SearchGrowthPerSec,

SearchRadiusMin, SearchRadiusMax);
```

That is the whole "expanding circle" model — a straight line starting at 5m, growing 0.6m every second, capped at 18m:

Time since last confirmed sighting	Search radius
t = 0s (just lost them)	5.0m
t = 5s	8.0m
t = 10s	11.0m
t = 15s	14.0m
t = 20s	17.0m
t = 21.7s	18.0m (hits the cap — stays here)
t = 45s	18.0m -> hunt budget expires, squad stands down together

The search area reaches its maximum size in under 22 seconds, then plateaus for the rest of the 45-second budget rather than growing forever.

Worked example, start to finish:

1. t=0s: Guard_A confirms the trainee at position (20,0), heading north. Squad.ReportConfirmedContact fires — the "write to the shared blackboard" moment. PointLastSeen=(20,0), LastSeenHeading=north, hunt clock resets to 0.
2. t=0s (same instant): FanOutSearch runs its "wave 0" — since the heading is known, every available searcher sweeps forward along it, side by side, covering the width of the escape route (LeadBase=5m ahead, spaced LateralStep=3.5m apart per searcher) — a directed chase, not a blind circle.
3. t=8s: nobody found them. A searcher's sweep comes up empty, calls Squad.NotifySearcherExhausted. The squad checks: still inside the 45s budget? Yes -> re-tasks that one searcher to a fresh point, now at radius approximately 5+8x0.6=9.8m around the last-seen point, rather than sending the whole squad home.
4. t=8s, wave 1+ instead of wave 0: since the direct chase already came up empty once, subsequent re-tasking switches from "everyone sweeps forward" to "divide and cover every angle" — bearings fanned at 0 degrees, +/-60, +/-120, +/-180 around the escape direction (LaneAngle), so the squad checks the sides and eventually behind, in case the trainee doubled back.
5. t=45s: still nothing. HuntShouldContinue() now returns false — the budget is spent. Squad.EndHunt() fires, and EVERY non-guardian squad member is told to StandDown() in the same instant — not one NPC quietly giving up while the others keep hunting.
6. Any time before t=45s, if ANY squad member gets a fresh sighting, ReportConfirmedContact fires again — the clock resets to 0, the radius collapses back to 5m, and everyone re-converges on the new point instead of continuing to search the old, stale area.

That reset-on-resighting behaviour is the direct code implementation of "a fresh sighting resets the anchor and collapses the radius, causing searchers to re-converge" — the search-theory line from Section 4 of the literature review.

7.5 Squad support brain (independent of Leader directives)

Non-guardian, non-engaged members also run a lightweight local decision each tick: HELP an injured/dead ally (health <= allySupportHealthThreshold=60), SUPPORT an ally who currently has eyes on the player (flank beside them), or SWEEP (trigger FanOutSearch so searchers spread rather than clump). This runs alongside the Leader directive system, not instead of it.

7.6 Quick reference — every squad-coordination number in one table

Constant	Value	What it controls
Ring 1 delay	0.3s	Squad-wide alert propagation
Ring 2 delay	1.0s	Nearby non-squad NPCs going suspicious
Ring 2 radius	15m	How far Ring 2 reaches
Standoff distance (Converge)	3.5m	How close squad gets before stopping

Constant	Value	What it controls
Flank offset	5m	How far each member swings to the side on Engage
Hunt budget	45s	Total time squad presses a lost contact
Search radius, start	5m	Radius the instant contact is lost
Search radius growth	0.6m/sec	How fast the search circle expands
Search radius cap	18m	Maximum size reached (~t=21.7s)
Fan-out cooldown	4s	Minimum gap between whole-squad re-taskings
Escape-direction memory	12s	How long a reported "they went that way" stays trusted

None of these are literature-derived — same honest-gap status as the role bonuses (Section 6.6) and stated plainly in Section 12 of the literature review: doctrine gives no exact number for lost-contact persistence, so 45s is an engineering judgement, not a citation.

8. Telemetry (TelemetryLogger.cs)

Four record types, logged in real time and written to JSON at session end (Application.persistentDataPath/Telemetry/): StateChanges (every FSM transition), Decisions (every NPCSelector.SelectBest call, with score breakdown), Snapshots (periodic position/state samples), Directives (every Leader Converge/Flank/Hold command). OnStateChangeLogged is the C# event hook Module 4's AAR/dashboard subscribes to, so the timeline can be rebuilt without Module 2 needing to know Module 4 exists.

9. Literature Grounding — Full Citation Map

9.1 From the literature review body (Literature_Review_Module2.docx)

- U.S. Army Battle Drill 2 ("React to Contact") — CALL Handbook 96-3: on contact, the unit returns fire immediately and takes the nearest cover, NEVER withdraws as a reflex; "Break Contact" is a separate, deliberate drill. Directly motivated the no-casual-give-up fix (Section 4.3) and the shared hunt (Section 7.4).
- FM 3-21.71 (Mechanized Infantry Platoon and Squad), Appendix E — Battle Drills: shared situational awareness, the ADDRAC fire-command format, leaders converging reinforcements onto contact. Modelled as the squad blackboard.
- Phillips, K., et al. (2014). "Wilderness Search Strategy and Tactics." Wilderness & Environmental Medicine, 25(2), 166-176. Point Last Seen / Last Known Point and the time-dependent expanding search area — modelled directly in the growing search radius (Section 7.4).
- Orkin, J. (2006). "Three States and a Plan: The AI of F.E.A.R." GDC / Monolith Productions. The two-layer GOAP squad architecture (autonomous per-agent behaviour beneath a squad coordination layer) — the closest published computational precedent to TerroristController + Squad.
- The Dupuy Institute (2018). "Breakpoints in U.S. Army Doctrine." Unit cohesion collapse driven by leadership loss, casualties, surprise — not a raw casualty count. Modelled as the _escalation accumulator + CombatPosture flip (Section 4.4).

- U.S. Army (1996), FM 3-06.11, ATP 3-21.8, FM 7-8 — battle-drill and urban-clearing doctrine underpinning the base-of-fire/manoeuvre-element and systematic-clearing design intent.
- NIST glossary, "Point Last Seen (PLS)" — the search-theory anchor concept.
- Thesis (2025), DiVA digital repository, diva2:1972169 — corroborates Orkin's two-layer GOAP description.

Honest gaps the review states plainly (Section 12 of the literature review): no source addresses an irregular/semi-trained defender specifically; no source quantifies a lost-contact persistence duration (the 45s hunt budget is a tuning decision informed by, not fixed by, the literature); the hostage-guard behaviour is a derivation, not a citation; the expanding-circle search model is an idealisation that ignores interior geometry.

9.2 From the NPCSelector formula postscript (added Aug 2026 to the same document)

- Mark, D. (2009). Behavioral Mathematics for Game AI — the foundational multi-factor utility-scoring text.
- Dill, K. (2010). "Improving AI Decision Making Using Utility Theory." GDC.
- Dill, K. & Martin, L. (2011). "A Game AI Approach to Autonomous Control of Virtual Characters." I/ITSEC — utility-based AI for a military-training virtual character (same application domain as this project; see the honest strength caveat in Section 6.1.6 of this document).
- Dill, K. (2012). "Design Patterns for the Configuration of Utility-Based AI." I/ITSEC.
- Lewis, M. (2015). "Choosing Effective Utility-Based Considerations." Game AI Pro 3, ch.13 — names distance and line-of-sight as standard utility-AI scoring considerations.
- ACM Computing Surveys (2024). "A Systematic Literature Review on Multi-Robot Task Allocation" — distance + capability + availability as the canonical MRTA factor set.
- Weapon-Target Assignment problem (operations research) — distance + capability constraints in military assignment (see honest strength caveat, Section 6.1.5).
- Computer-Aided Dispatch (CAD) industry documentation — real-world dispatch practice, not academic evidence (see honest strength caveat, Section 6.1.4).

9.3 Independently verified during this conversation, with links

These sources were checked directly (not taken on trust from the literature review) specifically to verify and, where needed, correct the NPCSelector formula claim (Section 6):

- Weighted sum model — Wikipedia: https://en.wikipedia.org/wiki/Weighted_sum_model
- Weight stability intervals for the weighted sum model — ScienceDirect: https://www.sciencedirect.com/science/article/pii/S0957417425020792
- Triantaphyllou, E. (2000). Multi-Criteria Decision Making Methods: A Comparative Study — https://www.scribd.com/document/542591911/Applied-Optimization-44-Evangelos-Triantaphyllou-Auth-Multi-criteria-Decision-Making-Methods-a-Comparative-Study-Springer-US-2000
- Simple Additive Weighting (SAW) normalisation formula: https://media.neliti.com/media/publications/326766-simple-additive-weighting-saw-method-in-f8f093e8.pdf
- Lewis, M., "Choosing Effective Utility-Based Considerations," Game AI Pro 3: https://www.gameaipro.com/GameAIPro3/GameAIPro3_Chapter13_Choosing_Effective_Utility-Based_Considerations.pdf

- Dill & Mark, GDC 2010 slides, "Improving AI Decision Modeling Through Utility Theory": https://media.gdcvault.com/gdc10/slides/MarkDill_ImprovingAIUtilityTheory.pdf
- Dill, K., "Design Patterns for the Configuration of Utility-Based AI": https://course.ccs.neu.edu/cs5150f13/readings/dill_designpatterns.pdf
- Dill & Martin (2011), "A Game AI Approach to Autonomous Control of Virtual Characters": https://course.ccs.neu.edu/cs5150f13/readings/dill_granny.pdf
- Weapon-target assignment problem — Wikipedia: https://en.wikipedia.org/wiki/Weapon_target_assignment_problem
- Exact and heuristic algorithms for the WTA problem — Springer: https://link.springer.com/article/10.1007/s10479-022-04525-6
- A Systematic Literature Review on Multi-Robot Task Allocation — ACM Computing Surveys, 2024: https://dl.acm.org/doi/10.1145/3700591
- CAD dispatch software guide — Resgrid: https://blog.resgrid.com/cad-dispatch-software/
- Utility AI normalisation walkthrough (practitioner explainer): https://yggdrasil-917.github.io/posts/utility-ai/utility-ai/

10. Viva Quick Reference

"How do you prioritise events?"

Two layers: broadcast-vs-targeted routing decides whether everyone reacts or one NPC is chosen; for targeted events, NPCSelector scores every candidate on distance/role/state/LOS and the highest scorer responds. There is no global priority queue — "priority" is encoded in WHO responds, not in event ordering.

"Is this from a paper?"

The formula is custom, but the METHOD (weighted multi-criteria scoring) is a real, formally-named technique (Weighted Sum Model / Simple Additive Weighting) with direct precedent in published military training-simulator AI. The current implementation was checked against that formal definition and found to skip its required normalisation step for two of four terms — a real, fixable gap, not fatal to the underlying technique.

"How do you know your scoring is better than nearest-only?"

The ablation experiment: same events, multiple configurations with one factor zeroed out at a time, compared against the full formula. The results (documented in MODULE2_ABLATION_STUDY_REPORT.md) show each factor genuinely changes the outcome — this proves each factor is doing real work, not that the resulting decisions are doctrinally correct; those are different claims and only the first is being made here.

"Why is the role bonus 3.0 and not some other number?"

No literature source fixes that number — say so directly. Defend it structurally instead: it is capped so role fit cannot single-handedly override distance/state/LOS (bounded influence, not an absolute rule), and the relative ordering of the three bonus values is the load-bearing design choice, not their absolute magnitude.

"What about leader coordination?"

Leader -> Alert issues Converge; Leader -> Engage issues Flank (each member computes its own flank side via offset-perpendicular geometry, so the squad splits naturally around the threat while the leader holds the front). Demonstrable live: spawn a Leader + 2 squadmates on the same squadId, alert the Leader, watch convergence, then engagement -> fan-out.

"What is the actual research contribution here, not just engineering?"

Three things the literature review states are not documented elsewhere: (1) fully autonomous event-driven NPCs — both hostile and civilian — with no instructor scripting; (2) a single shared event model driving both populations plus coordination; (3) context-aware multi-factor responder selection with ablation evidence quantifying each factor's contribution.

11. Evaluation Results — Ablation Study Diagrams

The diagrams below are the actual evaluation-results evidence for Section 6 (does each NPCSelector factor genuinely matter, and does role-bonus magnitude track measured dominance). These are the ablation-study RESULT charts only — the weight-SENSITIVITY charts (each weight swept 0-4x the default; MODULE2_WEIGHT_SENSITIVITY_REPORT.md) are a separate study and are intentionally not included here. Full method for every chart below is in MODULE2_ABLATION_STUDY_REPORT.md (Sections 1-6 for GunshotHeard, Section 7 for RoomBreached/AllyDownSeen).

11.1 GunshotHeard (favours Roamer, bonus 2.0) — the original study

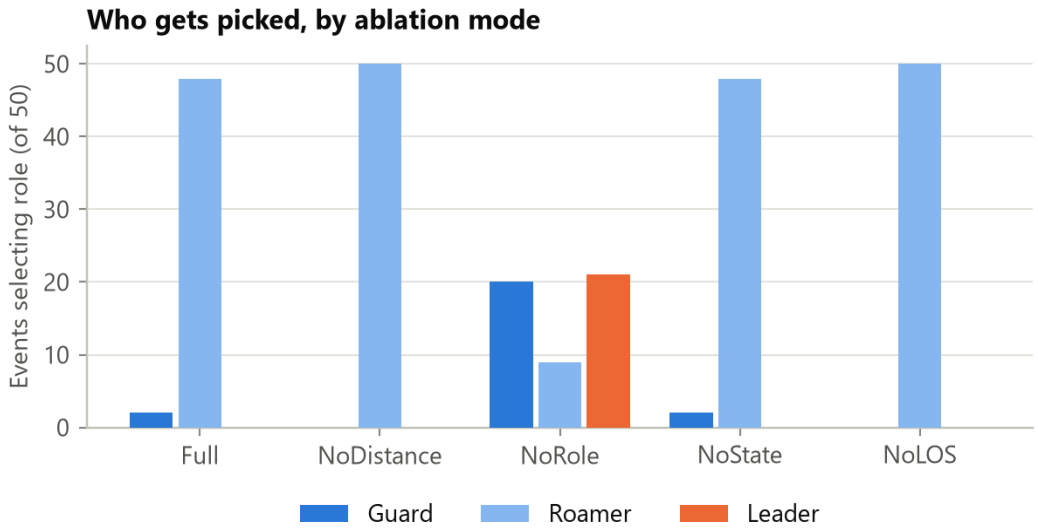


Figure 1. Which role gets picked, by ablation mode, for GunshotHeard events. Roamers win almost every event under Full/NoDistance/NoState/NoLOS; only removing Role itself changes the picture.

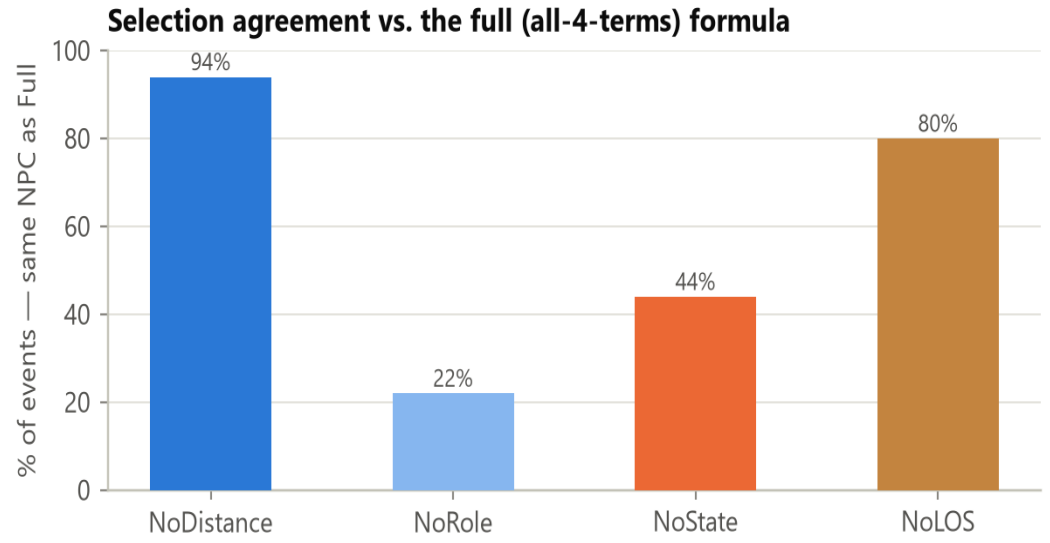


Figure 2. % of GunshotHeard events where each ablated mode picked the same NPC as the full formula — the headline ranking of how much each term matters: Role (22%) > State (44%) > LOS (80%) > Distance (94%). Lower % = that term matters more.

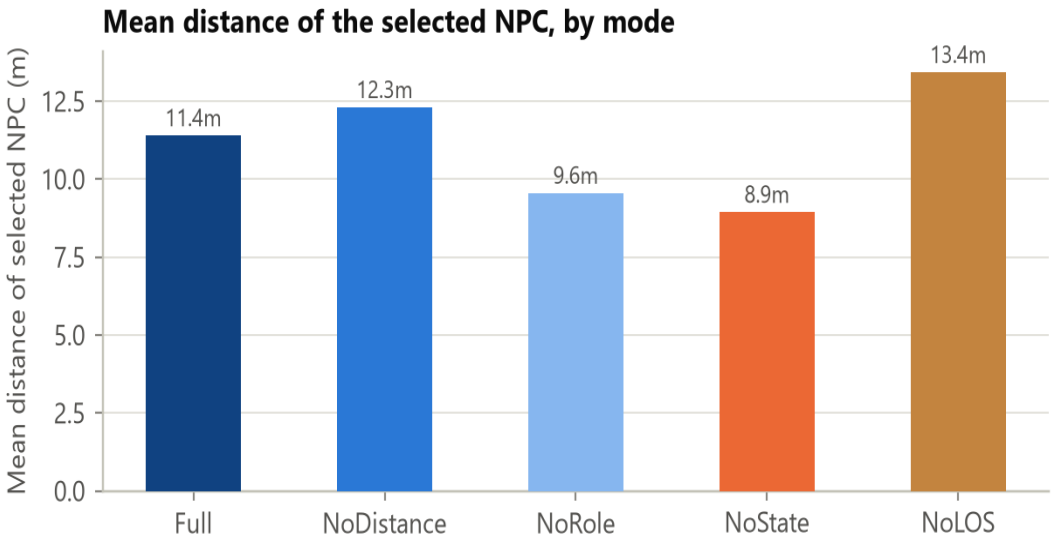


Figure 3. Mean distance of the selected NPC, by mode. The only valid pairwise comparison is Full (11.4m) vs. NoDistance (12.3m) — removing distance sends a responder about 0.9m farther away on average.

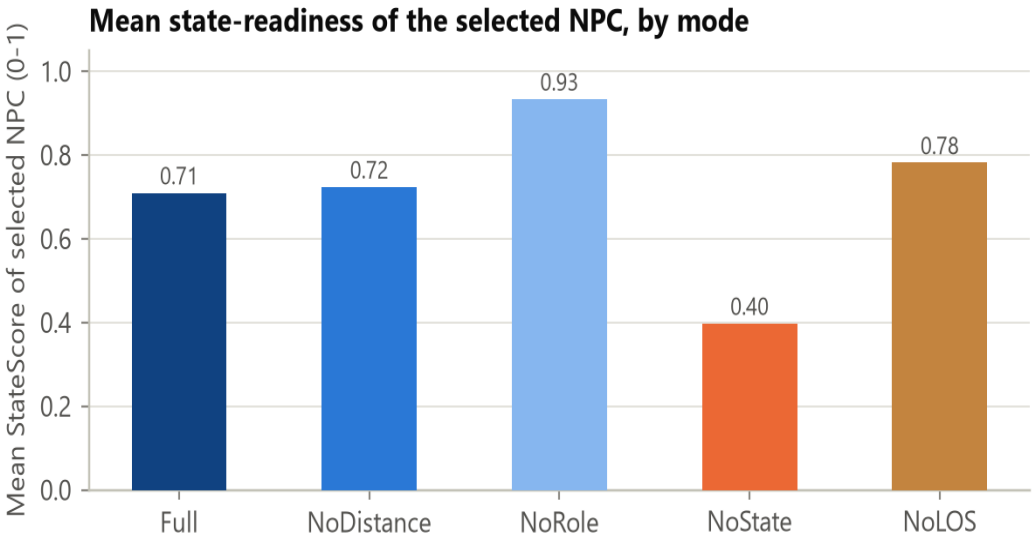


Figure 4. Mean state-readiness of the selected NPC. Full (0.71) vs NoState (0.40): removing the state term nearly halves the average readiness of who gets sent.

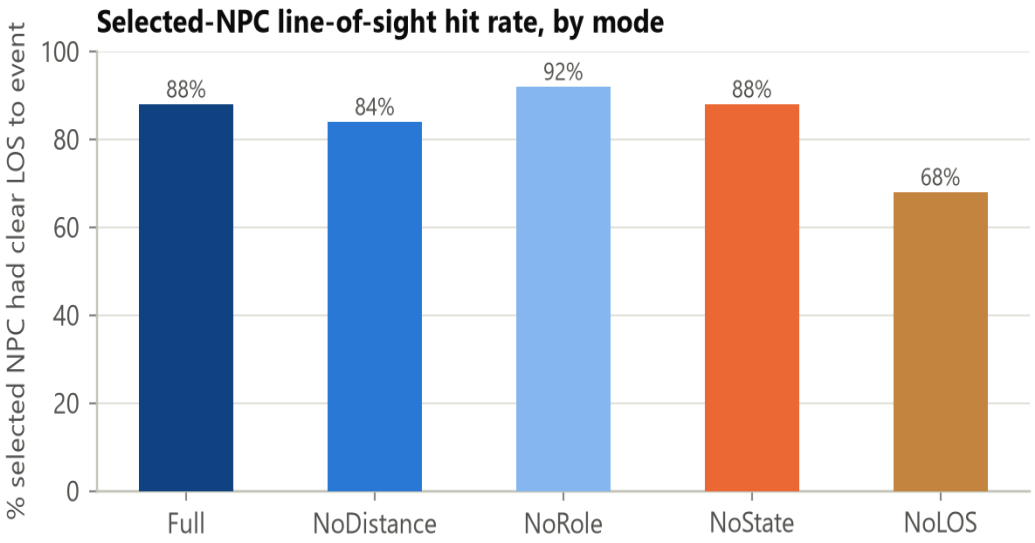


Figure 5. % of selections with clear line-of-sight to the event. Full (88%) vs NoLOS (68%): without the perception term, the selector sends a "blind" responder almost 3x more often.

11.2 RoomBreached (favours Guard, bonus 3.0)

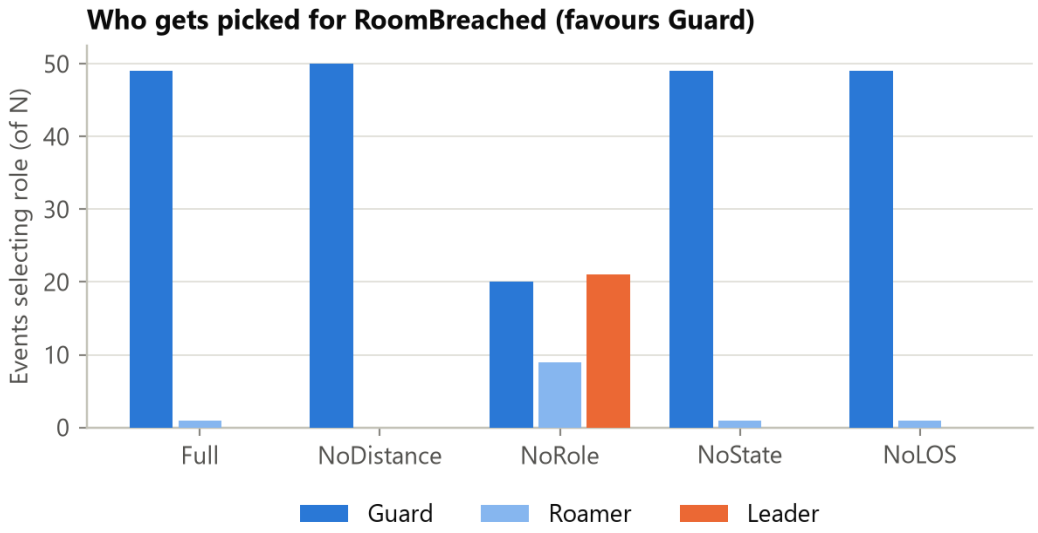
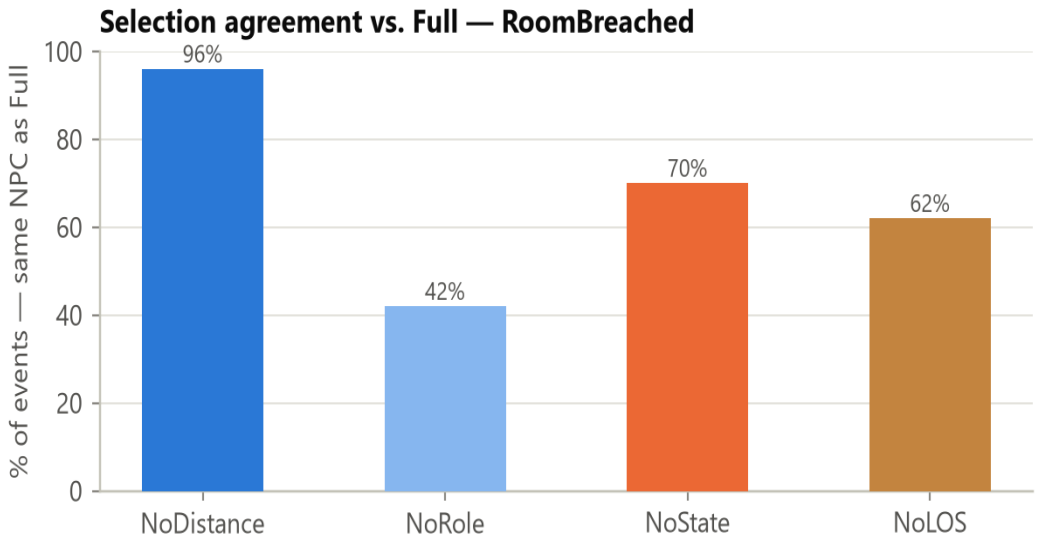


Figure 5a. Who gets picked for RoomBreached (favours Guard). Guard wins 49/50 events under Full — the same dominance pattern the original study found for Roamer under GunshotHeard, now confirmed for a second role-bonus entry.



Selection agreement vs. Full — RoomBreached. NoRole drops agreement to 42%, the largest swing of the three event types, consistent with Guard carrying the largest bonus (3.0) of the three role entries.

11.3 AllyDownSeen (favours Leader, bonus 1.5)

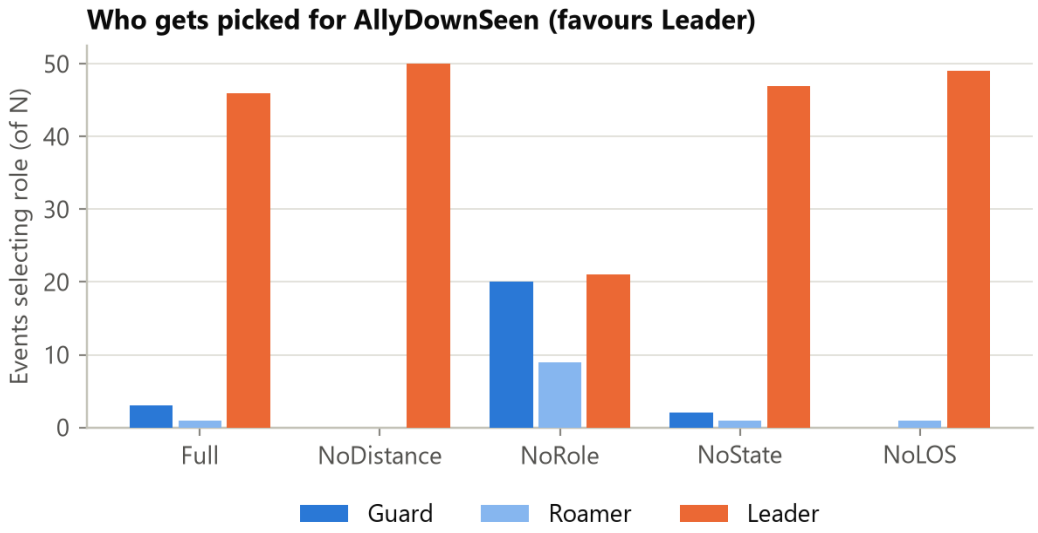
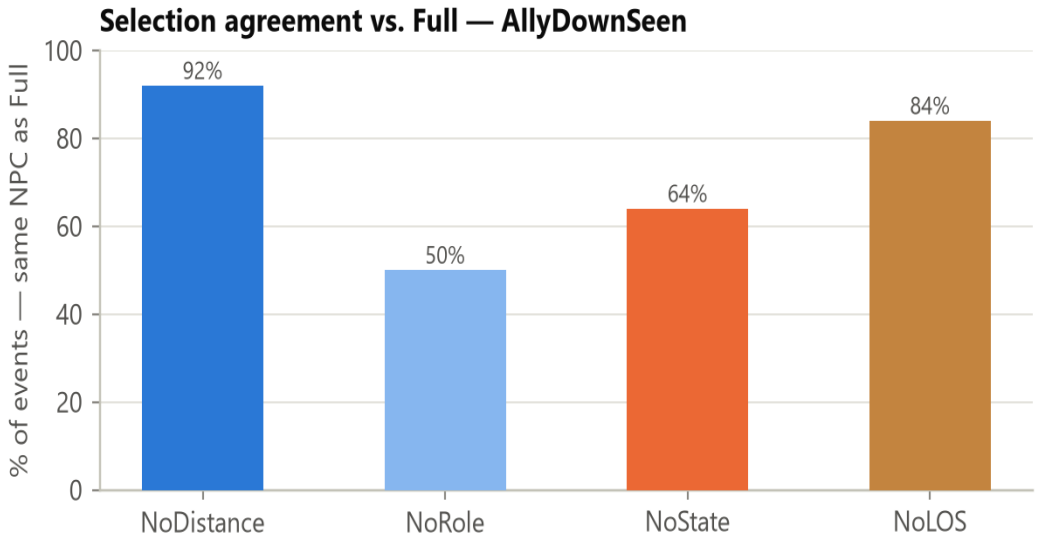
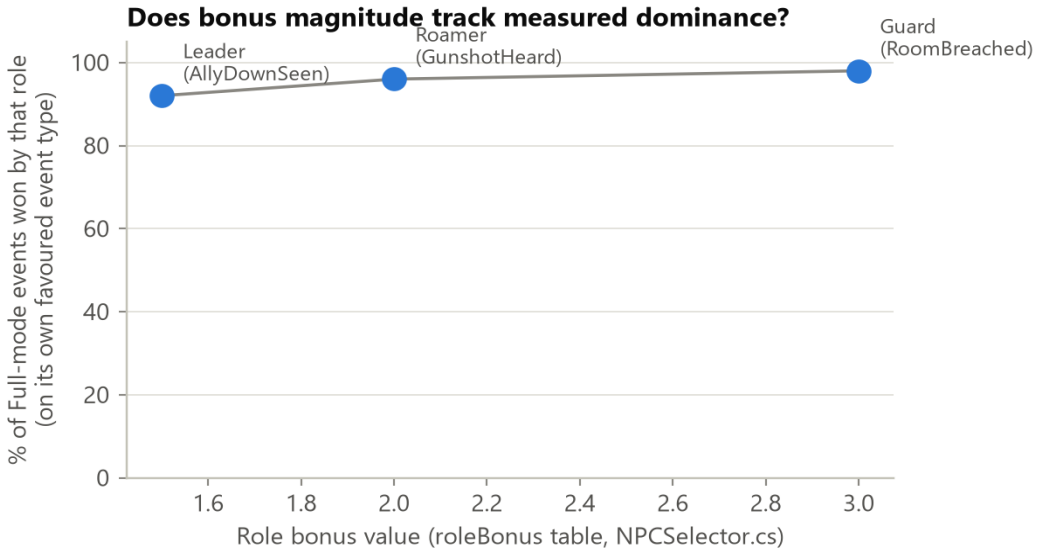


Figure 5b. Who gets picked for AllyDownSeen (favours Leader). Leader wins 46/50 events under Full.



Selection agreement vs. Full — AllyDownSeen. NoRole drops agreement only to 50%, the smallest swing of the three event types, consistent with Leader carrying the smallest bonus (1.5) of the three role entries.

11.4 Does bonus magnitude track measured dominance?



For each event type, the % of Full-mode events won by that event's favoured role, plotted against that role's bonus value. The relationship is monotonic — Guard (3.0) 98%, Roamer (2.0) 96%, Leader (1.5) 92% — real evidence the RELATIVE ordering of the three bonus magnitudes produces the intended relative behaviour (Section 6.5), even though the absolute values are still not literature-derived (Section 6.6).

This document intentionally excludes the ablation-chart-GENERATION process (the live Unity-MCP run extending the ablation study to RoomBreached and AllyDownSeen events, and how each figure above was produced) — that method and full commentary is documented separately in MODULE2_ABLATION_STUDY_REPORT.md (Section 7) and MODULE2_EVALUATION_BRIEFING. Section 11 above reproduces the RESULT diagrams themselves for a single self-contained reference.